

Speed vs. Scale: Technical and Economic Trade-offs in Stream Analytics Using Spark and Hadoop

Lim Yu Han
Faculty of Computing
Universiti Teknologi Malaysia
Johor Bahru, Malaysia
yuhan@graduate.utm.my

Abstract—This study evaluates the efficacy of Apache Spark Streaming (Spark) as a high-performance alternative to Hadoop MapReduce (Hadoop), focusing on the critical balance between technical speed and economic scale. As data velocity accelerates via global online platforms, conventional disk-based batch-processing frameworks like Hadoop fail to provide the sub-second responsiveness required for modern decision-making. This study analyzes the architectural mechanics of Spark, specifically its discretized stream (DStream) model and in-memory computation. A comparative analysis reveals that Spark achieves a performance gain of up to 100 times for iterative workloads by eliminating the I/O bottlenecks inherent in the Hadoop Distributed File System (HDFS). Furthermore, a case study of Riot Games demonstrates the practical necessity of low-latency processing, exemplified by Riot Games’ use of over 200 nodes processing 500 billion daily events for maintaining game integrity. The findings conclude that while Spark is the definitive solution for high-velocity hot data, its high Total Cost of Ownership (TCO) and intensive RAM requirements make Hadoop the economically superior choice for Petabyte-scale historical archives.

Index Terms—Apache Spark Streaming, Stream Processing, Real-Time Data Analytics, Hadoop MapReduce, In-Memory Computing

I. INTRODUCTION

Big data, stemming from social media, the Internet of Things (IoT), and other global online platforms, is driving a paradigm shift in data processing needs. In the era of modern applications, the data used for analysis are no longer limited to historical data, they must be immediately available for real-time analysis, provide actionable insights, and be used in high-stakes environments like financial fraud detection, network security, and real-time user behavior analysis (Zaharia et al., 2016). In these areas, the time from data creation to data insight, also known as latency, needs to be as quick as possible to avoid any potential financial consequences or security violations. However, traditional data processing methods are often technically inadequate to process these high-velocity data streams effectively (Lopez et al., 2016).

Hadoop MapReduce (Hadoop) was the standard of data analysis for large-scale data in the past (Shi et al., 2015). It is based on a distributed batch-processing model to manage large and static data sets. But Hadoop is essentially limited by the architecture that is disk-based. Each processing step involves writing intermediate results to physical disks, adding substantial latency and making it impractical for real-time

applications. Verma et al. (2016) pointed out that Hadoop was created for at-rest data and does not have the persistent execution environment required to process in-flight continuous streams.

These limitations have led to the development of stream analytics as a specialized area of high performance data processing (Lopez et al., 2016). In-memory computation and micro-batch processing are supported by frameworks like Apache Spark and its extension, Spark Streaming, which facilitates efficient analytics. These frameworks enable organizations to handle data processing near real-time by moving the computation workload from I/O to distributed RAM.

In this study, Apache Spark Streaming (Spark) is thoroughly evaluated technically. It introduces the architecture of the discretized streams (DStreams); shows how its performance (speed, latency and scalability) compares with the Hadoop baseline; and validates this analysis with a real-world example of Riot Games. The purpose of the study is to show that contemporary stream analytics frameworks address some of the structural shortcomings of the traditional batch-processing approach and are capable of satisfying the requirements of the modern data landscape.

II. LITERATURE REVIEW

A. Overview of Stream Analytics with Spark Streaming

Apache Spark Streaming is a scalable extension of Apache Spark that allows for stream analytics on continuous data streams. Stream analytics is the process of analysing a high velocity of data in real time to gain insights that can be used for action, which is crucial to modern applications like fraud detection, recommendation systems and IoT monitoring (Nasiri et al., 2019).

B. Architecture and Working Mechanism

In the Spark model, continuous data streams are processed in a micro-batch model, which allows for high-performance analytics. The technical essence of this architecture is Discretized Stream (DStream), which is a series of Resilient Distributed Datasets (RDDs) (Zaharia et al., 2013). Spark is different from Hadoop, which was designed to scale to thousands of commodity nodes with HDFS for high throughput, but is designed to have sub-second latency. This renders Spark as the hot data solution for quick decision making, while Hadoop

is the economical solution of cold data storage at exabyte scale (Shi et al., 2015). The structure of these components, from ingestion to processing engine, is shown in Fig. 1.

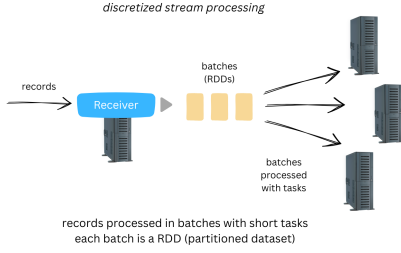


Fig. 1. Architecture of Spark for real-time data processing

The data flow follows a sophisticated lifecycle:

- **Ingestion & Replication:** Data is ingested via receivers from sources like Kafka. To ensure high availability, the receiver replicates the incoming data to other worker nodes before it is even processed.
- **Discretization:** The streaming context partitions the stream into time-based intervals (e.g., 500 ms). Each interval's data is bundled into an RDD.
- **Parallel Execution:** The Spark engine executes a Directed Acyclic Graph (DAG) of transformations. Because the data is already partitioned, these transformations occur in parallel across the cluster's cores, maximizing throughput.
- **Unified Pipeline:** A major architectural advantage is the ability to join streaming data with static historical data stored in Hive or Parquet, allowing for complex join operations that enrich live streams with historical context (Armbrust et al., 2018).

C. Design Principles and Performance Characteristics

In-Memory Computing is the foundation of the performance of Spark. Spark eliminates the disk I/O penalty that bedevils Hadoop (Zaharia et al., 2016) by keeping intermediate RDDs in the distributed RAM memory of the cluster. This is particularly important for iterative algorithms that reuse the same data.

One of the key design principles is fault tolerance through lineage. A distributed system in a traditional way will require a rollback to a checkpoint on the disk in case of node failure. Spark on the other hand, keeps a trace of every RDD. In the event that a partition is lost, only the portion of data from the original source is recomputed using the lineage graph. The lazy recovery model is a recovery model that keeps the system performing effectively during hardware failures. This dependency tracking is visualized in the lineage graph presented in Fig. 2, which maps the transformation stages.

In addition, dynamic load balancing can be used to redistribute tasks between nodes (stragglers) that are processing

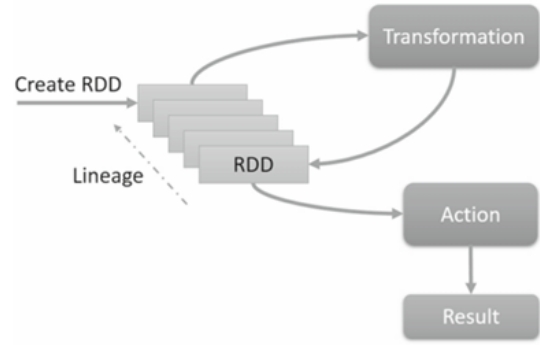


Fig. 2. RDD Lineage Graph showing the Directed Acyclic Graph (DAG) of dependencies for fault-tolerant recovery (Adapted from Salloum et al., 2016).

at a slower rate, so that the micro-batch finishes within its time limit (Cheng et al., 2018).

III. COMPARATIVE ANALYSIS

Comparing Spark with Hadoop shows a paradigm change in high-performance data processing. The main difference is their architectural style for data and state management.

Most of all, the computation model is superior to Spark. Unlike Hadoop, which is a batch processing system that stores intermediate results to the Hadoop Distributed File System (HDFS) after each Map or Reduce stage, Spark operates in memory and avoids the extra I/O latency incurred by writing intermediate results to HDFS in multiple stages of transformations (Li et al., 2016). For iterative workloads, like machine learning algorithms, Spark can be up to 100 times faster than Hadoop, according to benchmarking studies (Zaharia et al., 2016). For even simple data transformations, Spark is always able to offer 10x performance improvements, thanks to its ability to keep the hot data in RAM throughout the various stages of processing.

The most important factor in stream analytics is indeed latency. Hadoop is slow to start up because it creates a new Java Virtual Machine (JVM) instance for each and every task, which can take several seconds (Sharma & Kaur, 2019). Conversely, Spark is near real-time responsive as it breaks large batches of data into smaller micro batches ranging from 500ms to a couple seconds. Persistent executors run across the cluster which avoids the JVM startup bottleneck and allows Spark to process data in low latency manner as required by time sensitive applications such as financial fraud detection (Verma et al., 2016).

In addition, Spark has the ability to create stateful stream processing, which means it can keep track of information over multiple batches, a capability that is not technically possible in the stateless, independent-task nature of Hadoop. This feature is crucial in today's data environments where it is necessary to track long-running user sessions or historic trends.

A. Case Study in Speed: Riot Games

Riot Games is one of the best examples of a global data processing system. Riot implements a huge Spark ecosystem

to process more than 500 billion events a day, reaching up to millions of events per second (Sharma et al., 2023). Spark offered an alternative to the batch-window of legacy systems used for data processing for player behavior. Riot’s transition to a Spark-based architecture on clusters of more than 200 nodes reduced the time it took to get data to insights from several hours to less than 5 minutes. This 90

B. Case Study in Scale: The New York Times

While Riot Games exemplifies Spark’s speed for live telemetry, the New York Times serves as a landmark academic example of Hadoop’s economic utility for massive-scale archival. To digitize their entire historical collection that contains more than 130 years of articles, the organization used Hadoop’s MapReduce framework to process 11 million scanned articles into digital PDFs (Sharma & Kaur, 2019). This project was seen to be completed within 24 hours by leasing a temporary cluster of 100 virtual machines, and showcasing the ability of Hadoop for offline batch tasks (Luo et al., 2016). Utilizing an in-memory framework like Spark for such a massive at-rest dataset would have been significantly more expensive due to the high cost of RAM-intensive nodes. This illustrates how Hadoop’s scale architecture provides a specific benchmark for Return on Investment (ROI) in deep historical archives where sub-second latency is not a priority.

These economic priorities are further analyzed in the following critical discussion. A consolidated summary of these technical and economic metrics is presented in Table I.

TABLE I
TECHNICAL AND ECONOMIC COMPARISON BETWEEN HADOOP AND SPARK

Criteria	Hadoop MapReduce	Apache Streaming	Spark
Processing Model	Disk-based Batching	In-memory	Micro-batch
Execution Over-head	High (New JVM per task)	Low (Persistent Executors)	
Data Handling	Static/Historical Data	Continuous Streams	
State Management	Stateless	Stateful (DStreams)	
Hardware Economy	High (Runs on cheap HDD)	Low (Requires expensive RAM)	
Cost Efficiency	High (Petabyte-scale cold storage)	Low (High RAM cost per GB)	
Economic Use-Case	Long-term auditing or archiving	High-velocity hot data	

C. Critical Discussion of Trade-offs

While the performance gains of Spark are substantial, it is bounded by economic reality. Utilizing the Infrastructure Total Cost of Ownership (TCO) concept, it is evident that RAM-optimized cloud instances required for Spark execution typically cost 3 times to 5 times more per hour than the storage-optimized instances used for Hadoop (Shi et al., 2015). This helps to create a definite separation for a business. Spark serves as the “Speed” solution for live telemetry and real-time detection, whereas Hadoop remains the “Scale” solution for historical auditing and long-term compliance. Utilizing Spark

for 10 years of historical logs would result in a negative ROI compared to Hadoop’s cost-effective, disk-based archiving.

Furthermore, the brittle nature of Spark under memory pressure means that if a stream volume exceeds allocated RAM, the system spills to disk, effectively losing its performance advantage while still incurring the higher hardware costs. In a high-stakes environment like Riot Games, if the volume of the telemetry data stream exceeds the allocated cluster RAM, the system is forced to spill to disk (Zaharia et al., 2016). This creates a dangerous performance cliff for the business:

- **Latency Spikes:** After data is written to disk, Spark’s in-memory benefit is lost, and latency can rise from milliseconds to minutes.
- **Business Impact:** For Riot Games, this would mean that for thousands of players could be affected by anti-cheat detection in a way that allows bad actors to cause disruptions in match integrity before it’s fixed.

Spark, unlike Hadoop, does not have built-in fail-over capabilities for overflow, and needs careful memory tuning to prevent such failures. Finally, for ultra-low latency requirements such as high-frequency trading, Spark’s micro-batching interval may still be too slow, and the actual stream processors like Apache Flink could be more technically suitable.

IV. CONCLUSION

In this study, the importance of stream analytics in the current HPDP landscape has been discussed and how Spark is one such technology that can help in bridging legacy systems latency gap. With data velocity continuing to grow, Hadoop-based systems have become a specialized approach for processing historical batch data instead of a general approach. They use disk storage and have high overhead when creating new tasks for each data slice, which makes them less responsive for real time operations.

In contrast, Spark’s micro-batch model is a high-performance middle-ground solution that uses in-memory computation and persistent executors. The comparative analysis carried out in this study reveals that Spark is up to 100x faster than Hadoop for iterative tasks and can achieve sub-seconds latency as opposed to minutes. These technical benefits enable companies such as Riot Games to keep maintaining service integrity by using real-time telemetry.

However, this study also identifies one major economic trade-off: the TCO caused by Spark’s intensive RAM requirements. Spark’s performance becomes brittle if memory pressure is applied. When the volume of data in the stream is too large to fit into RAM, the system spills to disk, effectively nullifying its speed advantage over Hadoop. It is essential to recognize that the technical superiority of Spark is bounded by economic reality. While Spark dominates in speed, for Petabyte-scale historical archives, the cost of maintaining enough RAM to hold the entire dataset is often prohibitive. In these scenarios, Hadoop’s ability to run on inexpensive, high-capacity disk-based nodes provides a superior ROI for non-time-critical batch jobs.

Ultimately, it is a balancing act between speed and scale, whether to use Spark or Hadoop. Both Spark and Hadoop have their own strengths, with Spark offering high-speed performance for today's fast-paced games and fraud detection systems, and Hadoop being the foundation for massive economic data storage. Technical is not an invalid argument against economic and they can be combined into a hybrid ecosystem, with speed prioritized for live streams, and scale prioritized for historical archives.

REFERENCES

- Armbrust, M., Das, T., Torres, J., Yavuz, B., Zhu, S., Xin, R., & Ghodsi, A. (2018). Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark. <https://doi.org/10.1145/3183713.3190664>
- Cheng, D., Zhou, X., Wang, Y., & Jiang, C. (2018). Adaptive scheduling parallel jobs with dynamic batching in spark streaming. *IEEE Transactions on Parallel and Distributed Systems*, 29(12), 2672–2685.
- Li, R., Hu, H., Li, H., Wu, Y., & Yang, J. (2016). MapReduce parallel programming model: A state-of-the-art survey. *International Journal of Parallel Programming*, 44(4), 832–866.
- Lopez, M. A., Lobato, A. G. P., & Duarte, O. C. M. (2016). A performance comparison of open-source stream processing platforms. In *2016 IEEE Global Communications Conference (GLOBECOM)* (pp. 1–6). IEEE.
- Luo, J., Wu, M., Gopukumar, D., & Zhao, Y. (2016). Big data application in biomedical research and health care: A literature review. *Biomedical Informatics Insights*, 8, BII-S31559.
- Nasiri, H., Nasehi, S., & Goudarzi, M. (2019). Evaluation of distributed stream processing frameworks for IoT applications in Smart Cities. *Journal of Big Data*, 6(1), 52.
- Salloum, S., Dautov, R., Chen, X., Peng, P. X., & Huang, J. Z. (2016). Big data analytics on Apache Spark. *International Journal of Data Science and Analytics*, 1(3), 145–164.
- Sharma, K., Mohammad, F. H., & Parashar, D. (2023). Apache Spark in Riot Games: A Case Study on Data Processing and Analytics. *International Journal of Advanced Computer Science and Applications*, 14(7).
- Sharma, M., & Kaur, J. (2019). A comparative study of big data processing: Hadoop vs. spark. In *2019 6th International Conference on Computing for Sustainable Global Development (INDIACom)* (pp. 1073–1077). IEEE.
- Shi, J., Qiu, Y., Minhas, U. F., Jiao, L., Wang, C., Reinwald, B., & Özcan, F. (2015). Clash of the titans: MapReduce vs. spark for large scale data analytics. *Proceedings of the VLDB Endowment*, 8(13), 2110–2121.
- Verma, A., Mansuri, A. H., & Jain, N. (2016). Big data management processing with Hadoop MapReduce and spark technology: A comparison. In *2016 Symposium on Colossal Data Analysis and Networking (CDAN)* (pp. 1–4). IEEE.
- Zaharia, M., Das, T., Li, H., Hunter, T., Shenker, S., & Stoica, I. (2013). Discretized streams: Fault-tolerant streaming computation at scale. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles* (pp. 423–438).
- Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., ... & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65.
- Zhang, H., Chen, G., Ooi, B. C., Tan, K. L., & Zhang, M. (2015). In-memory big data management and processing: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 27(7), 1920–1948.